

HETIC • WEB3 • PROMOTION 2026

DOSSIER DE CONCEPTION

RobLaude

Robot d'assistance autonome pour les personnes à mobilité réduite en ERP

ÉPREUVE 1 • BLOC 1 : CONCEPTION

OBJET DU DOSSIER

Analyser le besoin, expliquer les arbitrages et formaliser une architecture cohérente, en distinguant la cible initiale, la réalisation observée et les améliorations encore nécessaires.

Travail individuel : Wissem

Titre RNCP 36146 • Concepteur Développeur de Solutions Digitales
Année 2026

SOMMAIRE

- 1. Résumé exécutif.....3
- 2. Contexte et problématique.....3
- 3. Parties prenantes, besoins et priorités.....4
- 4. Périmètre fonctionnel.....5
- 5. État de l'art et arbitrages.....6
- 6. Architecture de la solution.....8
- 7. Contrats de communication et données.....10
- 8. Spécifications fonctionnelles.....12
- 9. Réalisation et validation.....14
- 10. Contraintes, risques et limites.....15
- 11. Axes d'amélioration et évolutivité.....16
- 12. Traçabilité des besoins.....16
- 13. Conclusion et sources.....17

1. Résumé exécutif

RobLaude est un prototype de robot d'assistance destiné aux personnes à mobilité réduite dans les établissements recevant du public. Le projet répond à une situation simple : dans un bâtiment administratif, un usager peut dépendre d'un tiers pour déplacer un document ou récupérer un objet. La solution imaginée associe une interface web accessible, un serveur chargé des missions et un robot mobile capable de se repérer, de naviguer et de remonter son état.

Le choix central n'est pas une technologie isolée, mais la séparation des responsabilités. L'interface reste centrée sur l'utilisateur. Le backend contrôle les comptes, les points et le cycle des missions. MQTT transporte les commandes et la télémétrie entre le monde web et le robot. ROS 2 coordonne ensuite les capteurs, la navigation et l'actionnement. Cette organisation a permis de faire évoluer les composants sans créer de dépendance directe entre l'application et le matériel.

La navigation, la cartographie et plusieurs flux de communication ont été testés sur le robot réel. La récupération autonome d'un objet reste plus fragile : le bras et la pince répondent, mais la chaîne complète de perception, saisie et transport demande encore de la stabilisation. Ce dossier ne masque pas cet écart. Il l'utilise pour distinguer la conception cible de la validation obtenue.

RECOMMANDATION

Conserver l'architecture en couches et concentrer la prochaine phase sur la sûreté, la robustesse réseau, la validation de la saisie et la réduction de la charge embarquée.

Compétence	Ce que le dossier démontre
C1.1	Besoin, acteurs, objectifs hiérarchisés, contraintes et évolutions probables.
C1.2	Comparaisons ciblées, critères de décision et conséquences des arbitrages.
C1.3	Architecture, contrats, spécifications et diagrammes cohérents avec le projet.

2. Contexte et problématique

2.1 Une difficulté d'usage dans les ERP

Les mairies, hôpitaux et bâtiments administratifs regroupent des services dispersés dans des couloirs, sur plusieurs zones ou à différents étages. Pour une personne à mobilité réduite, un déplacement supplémentaire pour transmettre un document ou récupérer un objet peut représenter un effort important. L'alternative consiste souvent à solliciter un agent ou un accompagnant. Le problème ne se résume donc pas au transport : il touche directement l'autonomie de l'utilisateur et l'organisation du personnel.

RobLaude a été conçu comme un service de proximité dans le bâtiment. L'utilisateur formule une demande depuis son téléphone ou un poste web. Le robot rejoint un point connu, transporte le document ou tente de récupérer un objet, puis se rend à la destination. Pendant le trajet, l'utilisateur doit comprendre ce qui se passe et pouvoir arrêter le mouvement.

2.2 Enjeux métier

Enjeu	Traduction pour RobLaude	Conséquence de conception
Autonomie	Limiter le recours systématique à un tiers.	Parcours guidé et commandes accessibles.
Confiance	Rendre l'état de la mission compréhensible.	Suivi temps réel et statuts explicites.
Sécurité	Permettre une interruption immédiate.	Arrêt dédié et gestion de la pause.
Continuité	Éviter qu'une panne d'un écran bloque tout le système.	Séparation frontend, backend et robot.
Maintenabilité	Faire évoluer le prototype avec une petite équipe.	Standards existants et composants isolés.

2.3 Retombées attendues

Les retombées sont formulées comme des objectifs, car le projet académique n'a pas conduit d'étude économique en exploitation. La solution vise à réduire certains déplacements d'assistance, à fluidifier les échanges internes et à rendre le service plus inclusif. Elle pourrait également offrir au personnel technique une vision centralisée des missions et de l'état du robot.

LIMITE DE L'ANALYSE

Aucun gain financier chiffré n'est annoncé. Il faudrait une expérimentation en ERP, un volume réel de missions et des coûts de maintenance observés pour calculer un retour sur investissement crédible.

Sources projet : [R1] Cahier des charges RobLaude v1.2 ; [R2] Architecture canonique.

3. Parties prenantes, besoins et priorités

3.1 Acteurs

Acteur	Attente principale	Point de vigilance
Utilisateur PMR	Créer et suivre une mission sans assistance technique.	Lisibilité, navigation clavier et action STOP visible.
Agent d'accueil	Utiliser le service et accompagner un chargement manuel.	Compréhension immédiate des statuts.
Administrateur	Configurer points, objets et consulter l'historique.	Droits distincts et traçabilité.
Personnel technique	Diagnostiquer le robot et maintenir la carte.	Accès aux états sans exposer ROS 2 au public.
Robot	Exécuter une commande valide et remonter son résultat.	Ressources de calcul et réseau limités.

3.2 Hiérarchisation

Les fonctions n'ont pas le même poids. La sécurité et la capacité à arrêter le robot passent avant la richesse du suivi. La navigation et le transport de document forment le premier service complet. La récupération d'objet apporte davantage d'autonomie, mais elle dépend d'une chaîne plus risquée : caméra, détection, calibration, bras, pince et validation de la prise.

Priorité	Besoins
Critique	Arrêt d'urgence, refus d'une mission impossible, état du robot connu.
Haute	Créer une mission de transport, naviguer, suivre l'avancement.
Moyenne	Configurer les points, objets et cartes du bâtiment.
Évolutive	Récupération autonome fiable, multi-robot, déploiement multi-site.

3.3 Contraintes initiales

- Une équipe de deux personnes et dix-huit semaines, avec une expérience plus forte sur le web que sur ROS 2.
- Un robot réel équipé d'un Jetson Nano à quatre cœurs et 4 Go de mémoire, ce qui limite les traitements simultanés.
- Un réseau local dont les adresses peuvent changer, avec un robot qui doit rester joignable sans configuration manuelle permanente.
- Des capteurs et un bras dont les limites devaient être découvertes par des essais sur le matériel.
- Une interface destinée à des usages mobiles et à des personnes pouvant naviguer au clavier ou avec des dispositifs adaptés.

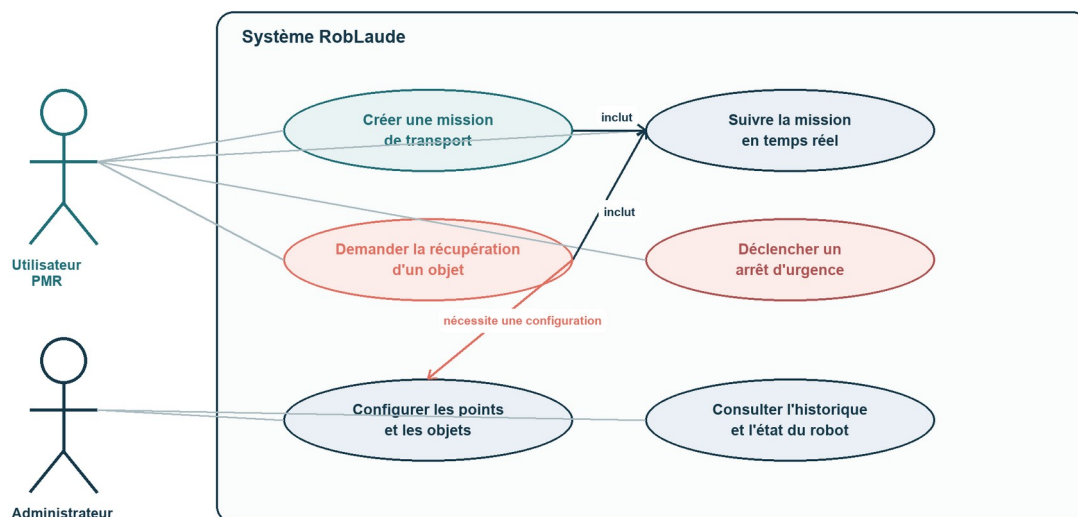
Sources projet : [R1] Cahier des charges ; [R2] Architecture ; [R3] Roadmap.

4. Périmètre fonctionnel

Le périmètre s'organise autour de sept cas d'utilisation. Quatre concernent directement l'utilisateur : transporter un document, récupérer un objet, suivre une mission et arrêter le robot. Trois relèvent de l'administration : configurer les points, définir les objets saisissables et consulter l'historique.

Périmètre fonctionnel de RobLaude

Acteurs, services attendus et dépendances principales



Lecture : l'administrateur possède aussi les droits d'un utilisateur.

Figure 1. Cas d'utilisation retenus pour le dossier de conception.

4.1 Périmètre minimal utilisable

Le premier scénario cohérent est le transport de document. Il mobilise déjà l'ensemble de la chaîne sans dépendre du bras : création de mission, attribution au robot, navigation vers le point de collecte, confirmation du chargement, navigation vers la destination et notification finale. Ce choix réduit le risque tout en validant l'architecture de bout en bout.

4.2 Extension vers la manipulation

La récupération d'objet reprend le même socle, puis ajoute la détection et la saisie. Elle n'est pas traitée comme un simple bouton supplémentaire. Elle impose des préconditions propres : objet connu, caméra disponible, bras calibré et stratégie de reprise en cas d'échec. Cette séparation évite qu'une fonction expérimentale fragilise le transport simple.

5. État de l'art et arbitrages

L'état de l'art a été limité aux décisions qui modifient réellement le coût, le délai, la performance ou la pérennité de la solution. Le but n'était pas de comparer toutes les bibliothèques disponibles, mais de choisir un chemin compatible avec l'équipe, le matériel et le besoin.

5.1 Interface : PWA ou application native

Critère	PWA	Application native
Accès	URL puis installation possible.	Installation via un écosystème mobile.
Développement	Une base React/TypeScript pour plusieurs écrans.	Développements et validations spécifiques par plateforme.
Matériel mobile	Accès suffisant pour le suivi et les formulaires.	Meilleure intégration aux API profondes du téléphone.
Choix RobLaude	Retenue : délai court, compétences disponibles, usage web central.	Non retenue : coût d'apprentissage sans bénéfice décisif ici.

La PWA ne signifie pas que le robot fonctionne hors ligne. Une mission nécessite le réseau. Le service worker sert surtout à l'installation et au chargement de l'interface. Cette limite est cohérente avec le besoin : masquer une perte de connexion pendant une commande robot serait trompeur.

Référence externe : [E1] MDN, Making PWAs installable. Source projet : [R1], section interface web.

5.2 Communication web : REST et WebSocket

REST convient aux opérations ponctuelles qui ont une réponse claire : créer une mission, consulter des points ou confirmer un chargement. Le suivi du robot produit au contraire des changements réguliers. Interroger le serveur en boucle augmenterait le trafic et le délai d'affichage. WebSocket permet au backend de pousser une position ou un statut dès qu'il le reçoit. Les deux mécanismes sont donc complémentaires.

5.3 Communication robot : MQTT ou exposition directe de ROS 2

Critère	MQTT entre web et robot	Accès direct à ROS 2
Découplage	Contrat stable entre deux environnements.	Le web dépend du graphe ROS et de ses types.
Réseau	Broker, reconnexion, QoS et état retained.	Découverte DDS plus sensible à la topologie réseau.
Sécurité	Un point de contrôle côté backend et broker.	Surface ROS 2 exposée au système web.
Évolutivité	Préfixe par robot et abonnements génériques.	Nommage et découverte à adapter pour chaque robot.
Choix	Retenu pour isoler le domaine web du domaine robotique.	Réservé aux communications internes du robot.

Références : [E2] spécification OASIS MQTT ; [E3] documentation ROS 2 sur topics, services et actions.

5.4 Framework robotique

Développer directement les drivers, la navigation, la localisation et les outils de diagnostic aurait dépassé le calendrier. ROS 2 Humble fournit un modèle par nœuds, des messages typés, des topics pour les flux continus et des actions pour les tâches longues. Nav2 expose notamment une action de navigation avec objectif, feedback et résultat, adaptée à un déplacement qui peut être annulé.

Nav2 et SLAM Toolbox ont donc été retenus plutôt qu'un moteur de navigation écrit sur mesure. Ce choix accélère le prototype et s'appuie sur un écosystème documenté. En contrepartie, il impose de comprendre les repères TF, les horloges, les QoS et le cycle de vie des nœuds. Le problème temporel des LiDAR a montré que cette complexité est réelle.

Références : [E3] ROS 2 Interfaces ; [E4] documentation Nav2 ; [R2] architecture RobLaude.

5.5 Hébergement des traitements

Option	Avantage	Limite	Décision
Tout sur le robot	Autonomie locale et faible dépendance au serveur.	Charge CPU et mémoire élevée sur le Jetson Nano.	Conserver la navigation critique ; limiter les flux et traitements non essentiels.
Tout sur un serveur	Plus de ressources de calcul.	Dépendance réseau trop forte pour les mouvements et capteurs.	Écartée pour le contrôle robot.
Architecture hybride	Métier côté backend, contrôle temps réel côté robot.	Contrats et synchronisation à maintenir.	Retenue.

5.6 Synthèse des recommandations

Décision	Coût et délai	Performance	Pérennité
PWA React	Une base pour mobile et desktop.	Interface légère ; dépend du réseau pour les missions.	Technologies web standard et source éditables.
MQTT	Ajoute un broker mais évite un couplage sur mesure.	QoS adapté à chaque flux ; télémétrie à limiter.	Contrats versionnés et extension multi-robot.
ROS 2 + Nav2	Courbe d'apprentissage, mais briques déjà disponibles.	Exécution locale ; réglages nécessaires sur Jetson.	Composants remplaçables derrière des interfaces ROS.
Docker sur Jetson	Réutilise le matériel livré malgré son OS d'origine.	Surcoût modéré en ressources.	Environnement reproductible et dépendances isolées.

6. Architecture de la solution

L'architecture repose sur quatre zones : l'expérience utilisateur, les services web, la messagerie et le robot. Une zone peut évoluer tant que son contrat avec la suivante reste stable. Ce principe répond directement à la taille de l'équipe et au caractère expérimental du matériel.

Architecture logique de RobLaude

Des responsabilités séparées, reliées par des contrats explicites

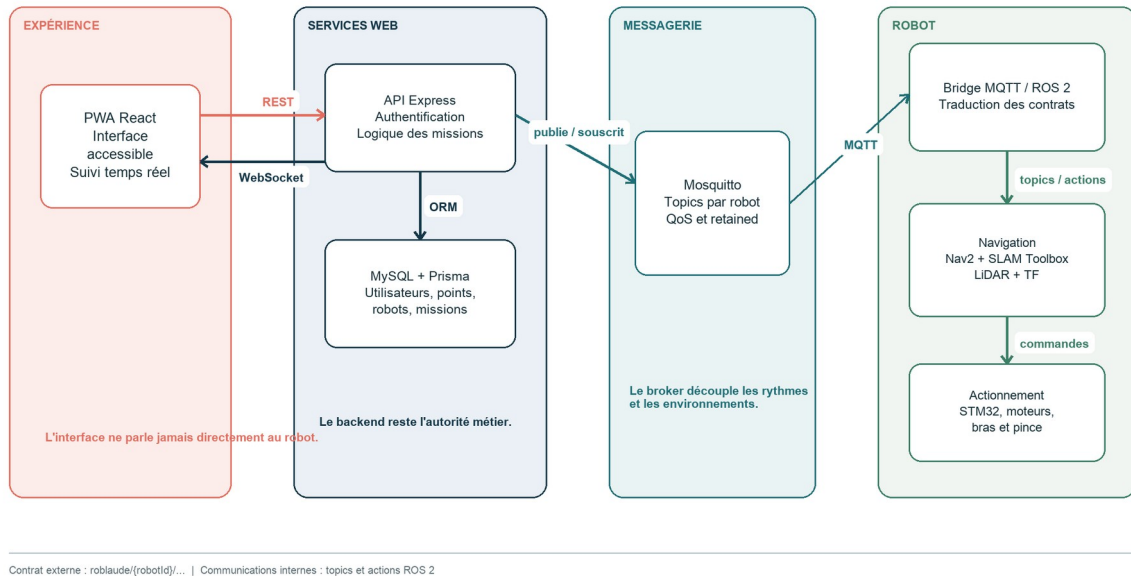


Figure 2. Architecture logique et responsabilités principales.

6.1 Frontend

La PWA présente les missions, les points et l'état du robot. Elle ne décide pas si une mission est possible et ne publie jamais sur MQTT. Cette limite volontaire empêche qu'une modification d'interface contourne les règles métier ou adresse directement des composants physiques.

6.2 Backend et base de données

Le backend Express authentifie les utilisateurs, valide les demandes et conserve l'état métier dans MySQL via Prisma. Il vérifie notamment la disponibilité du robot et les paramètres de mission avant de publier une commande. Il traduit ensuite les retours MQTT en mises à jour de données et en événements WebSocket.

6.3 Broker et bridge

Mosquitto reçoit les messages MQTT. Le bridge exécuté sur le robot traduit les commandes en messages ou actions ROS 2 et remonte la télémétrie. Le bridge ne remplace ni le backend ni les nœuds de navigation : il adapte deux modèles de communication différents.

6.4 Domaine robot

ROS 2 organise la partie embarquée en programmes spécialisés. Le nœud de mission pilote l'action NavigateToPose. SLAM Toolbox produit la carte et la relation entre les repères map et odom. Les drivers publient LiDAR, odométrie et batterie. Le STM32 reste responsable de l'actionnement bas niveau des moteurs et des servos.

6.5 Décomposition interne du robot

Composant	Responsabilité	Entrées	Sorties
mqtt_bridge	Adapter MQTT et ROS 2.	Commandes MQTT, topics ROS.	Topics ROS, télémétrie MQTT.
mission_executor	Orchestrer le scénario de mission.	Commandes internes, détections.	Objectifs Nav2, statuts, bras.
scan_restamper	Corriger le temps des scans fusionnés.	/scan_multi	/scan_fixed
SLAM Toolbox	Construire la carte et localiser.	Scans et TF.	/map et TF map → odom.
Nav2	Planifier et exécuter un déplacement.	Goal NavigateToPose, carte, obstacles.	Commandes de vitesse, feedback, résultat.
mapping_supervisor	Démarrer, arrêter et sauvegarder une cartographie.	Commandes de mapping.	État et résultat de sauvegarde.
STM32 / micro-ROS	Piloter le matériel bas niveau.	Vitesse, bras, pince.	Odométrie, IMU, batterie.

POURQUOI CETTE GRANULARITÉ

Chaque nœud a une responsabilité identifiable. Un problème de timestamp LiDAR peut être corrigé sans modifier le backend, la base de données ou l'interface.

6.6 Déploiement

Le Jetson Nano exécute ROS 2 Humble dans le conteneur m3pro. Le workspace RobLaude est monté dans ce conteneur et construit avec colcon. Un second conteneur héberge l'agent micro-ROS qui relie le STM32 au graphe ROS 2. Cette solution a permis de conserver le matériel livré tout en utilisant les versions logicielles nécessaires au projet.

Le backend, MySQL et Mosquitto peuvent être lancés avec Docker Compose sur la machine de développement. Le robot retrouve le broker sur le réseau local. L'adresse du robot étant attribuée par DHCP, un script le localise par son adresse MAC avant les opérations de déploiement.

Sources projet : [R2] Architecture canonique ; [R4] Spécification MQTT ; scripts de déploiement du dossier robot.

7. Contrats de communication et données

7.1 Arborescence MQTT

Chaque robot possède un sous-arbre `roblaude/{robotId}/...`. Cette convention évite de coder un ensemble de topics différent pour chaque machine et prépare la gestion de plusieurs robots. Les messages se répartissent entre commandes, télémétrie, état global et cycle de mission.

Famille	Direction	Exemples	Règle
cmd	Backend → Robot	mission, cancel, resume, emergency-stop	Événements non retained ; identifiant pour l'idempotence.
telemetry	Robot → Backend	position, batterie, carte	États périodiques ; fréquence adaptée à l'usage.
status	Robot → Backend	AVAILABLE, BUSY, ERROR	État retained pour une reprise rapide.
mission	Robot → Backend	ack, status, result	Accusé, progression et sortie clairement séparés.
connection	Robot → Backend	online true/false	Last Will pour signaler une rupture non propre.

7.2 QoS et nature des messages

Le niveau de QoS n'est pas identique pour tous les flux. Une position sera remplacée quelques instants plus tard ; la perte ponctuelle d'un échantillon est tolérable. Une commande de mission ou un résultat final exige davantage de garantie. Les états utiles à la reconnexion sont retained, tandis qu'une commande ne l'est jamais afin d'éviter son jeu involontaire.

Type	QoS retenu	Retained	Justification
Position	0	Oui	Flux fréquent ; la prochaine valeur remplace la précédente.
Batterie / état	1	Oui	Un nouvel abonné doit connaître le dernier état.
Commande	2	Non	Éviter perte ou doublon pendant une session, sans rejouer un ordre ancien.
Résultat final	2	Non	La clôture d'une mission doit être traitée une seule fois.

NUANCE TECHNIQUE

MQTT QoS 2 ne garantit pas qu'une commande publiée pendant une longue absence du robot sera toujours exécutée. Le backend doit aussi vérifier la présence et refuser une demande devenue incohérente.

7.3 Données métier

Le schéma Prisma matérialise les responsabilités du backend. Un utilisateur crée une mission. Une mission référence des points, un robot et éventuellement un objet. Les statuts représentent le cycle

de vie ; ils ne sont pas déduits uniquement de la dernière position reçue. Cette distinction permet de conserver un historique, d'expliquer un échec et d'éviter qu'une coupure réseau efface la demande.

Entité	Rôle	Relations principales
User	Identité et rôle.	Crée des missions.
Mission	Demande, type, statut et résultat.	Utilisateur, robot, points, objet éventuel.
Robot	État opérationnel et identification.	Exécute plusieurs missions dans le temps.
Point	Emplacement nommé et coordonnées.	Départ ou destination.
GraspObject	Objet configurable pour la saisie.	Associé aux missions de récupération.
MapSnapshot	Carte sauvegardée et métadonnées.	Liée aux sessions de cartographie.

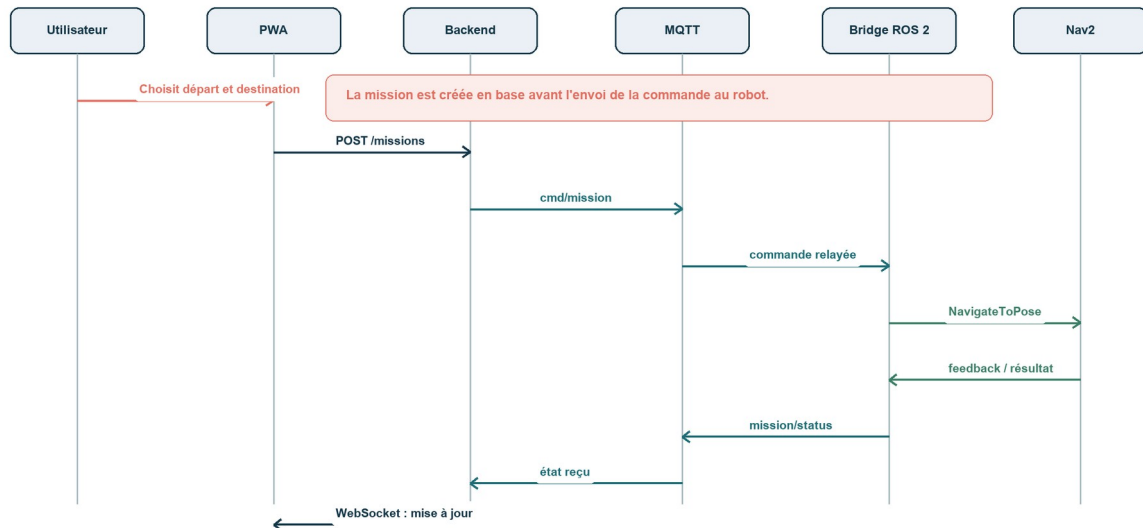
Sources projet : [R4] Spécification MQTT ; ``web/backend/prisma/schema.prisma``.

8. Spécifications fonctionnelles

8.1 Mission de transport

Séquence simplifiée d'une mission de transport

Du choix de l'utilisateur jusqu'au résultat de navigation



En cas d'arrêt d'urgence, une commande dédiée interrompt la navigation et place la mission en pause.

Figure 3. Flux simplifié d'une mission de transport.

Élément	Spécification
Préconditions	Utilisateur authentifié ; points configurés ; robot disponible et connecté.
Déclencheur	Validation du départ et de la destination dans la PWA.
Scénario nominal	Création en base, commande MQTT, navigation vers collecte, confirmation du chargement, navigation vers destination, résultat.
Suivi	Position et sous-état transmis au frontend via WebSocket.
Erreurs	Robot hors ligne, refus, timeout, obstacle bloquant, annulation ou arrêt d'urgence.
Postcondition	Mission terminée, échouée ou annulée avec un état et une raison enregistrés.

8.2 Arrêt d'urgence

Le bouton STOP doit rester accessible quel que soit l'écran. Son activation envoie une requête au backend, qui publie une commande dédiée. Le robot interrompt la navigation et renvoie un état de pause. L'utilisateur peut ensuite reprendre ou annuler. Cette logique évite de confondre une pause de sécurité avec un échec définitif.

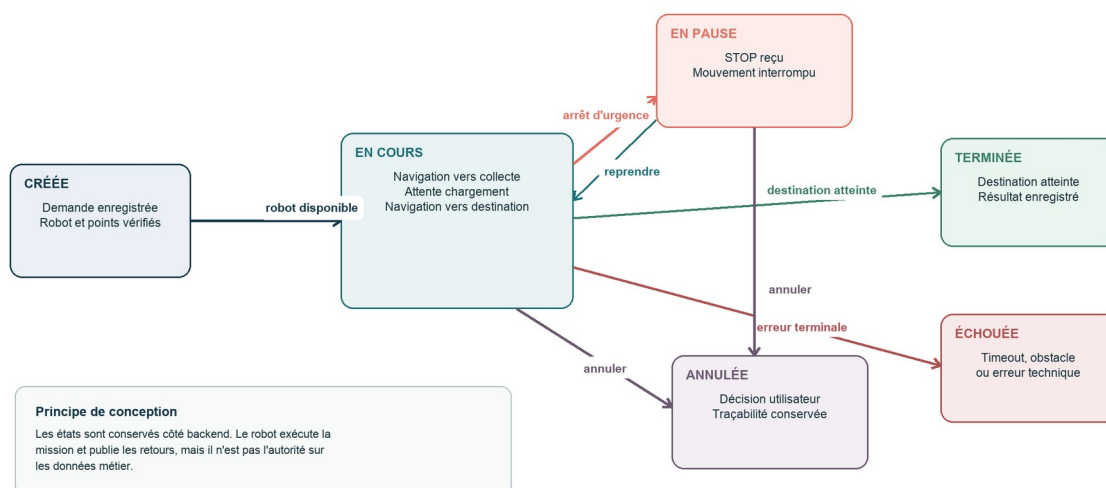
EXIGENCE DE SÛRETÉ

Le dispositif logiciel contribue à l'arrêt, mais ne remplace pas une chaîne d'arrêt d'urgence certifiée au niveau matériel. Une exploitation en ERP demanderait une analyse de risques et des validations réglementaires complémentaires.

8.3 Cycle de vie

Cycle de vie d'une mission

États principaux, reprise et sorties contrôlées



La récupération d'objet ajoute des sous-états de détection, saisie et dépôt, sans changer les états métier principaux.

Figure 4. États métier principaux d'une mission.

Le backend conserve l'état métier de référence. Le robot exécute et publie ses retours, mais une position ou un topic isolé ne suffit pas à redéfinir la mission. Cette règle facilite la reprise après une reconnexion et conserve une trace compréhensible pour l'utilisateur.

8.4 Récupération d'objet

Une mission de récupération ajoute quatre sous-étapes : rejoindre l'emplacement, détecter l'objet, effectuer la saisie puis transporter et déposer. Les erreurs de détection ou de prise doivent être distinguées d'une panne générale. Lorsque le robot reste opérationnel, l'utilisateur peut repositionner l'objet ou relancer une mission.

Sous-étape	Entrée	Résultat attendu	Sortie d'erreur
Détection	Couleur ou objet ciblé, image caméra.	Position exploitable de l'objet.	Objet non trouvé.
Approche	Position de l'objet.	Robot et bras correctement placés.	Position inaccessible.
Saisie	Pose du bras et commande pince.	Objet maintenu.	Prise échouée ou incertaine.
Transport	Objet saisi, destination valide.	Arrivée sans perte.	Objet tombé ou navigation

Sous-étape	Entrée	Résultat attendu	Sortie d'erreur
			impossible.
Dépôt	Destination atteinte.	Objet libéré et bras au repos.	Dépôt non confirmé.

9. Réalisation et validation

La présence d'un composant dans le dépôt prouve son implémentation, pas son fonctionnement sur le terrain. La validation ci-dessous sépare donc le code disponible, les essais documentés et les éléments encore fragiles.

Élément	État retenu	Preuve disponible
PWA et parcours de mission	Implémentés	Code React, tests frontend et runbook de démonstration.
API, données et MQTT	Implémentés	Code Express/Prisma, bridge et tests MQTT.
Navigation Nav2	Testée sur robot réel	Historique de sessions et goals atteints documentés.
Cartographie	Testée sur robot réel	Carte sauvegardée et procédure de cartographie.
Chaîne LiDAR / TF	Testée après corrections	Débits observés sans trou et absence d'erreurs temporelles dans l'état propre.
Bras et pince	Actionnement testé	Mouvements observés après correction du câblage.
Saisie autonome complète	Partiellement validée	Briques présentes, mais fiabilité de bout en bout insuffisante.
Arrêt d'urgence	Flux logiciel implémenté	Commande web/MQTT/ROS ; certification matérielle hors périmètre.

9.1 Un exemple de validation utile : le temps des scans

Les deux LiDAR publiaient des mesures correctes, mais le scan fusionné portait un timestamp situé environ une demi-seconde dans le futur. SLAM Toolbox cherchait alors une transformation TF à un instant qui n'existait pas encore. Les scans s'accumulaient dans une file puis étaient rejetés, empêchant la production cohérente de la carte.

Le nœud ``scan_restamper`` ne modifie ni les distances ni les angles. Il remplace uniquement le timestamp par l'heure ROS courante moins un décalage configurable de 0,2 seconde, puis republie sur ``/scan_fixed``. Le SLAM et les costmaps utilisent ce topic corrigé. Ce petit composant illustre l'intérêt de l'architecture modulaire : une contrainte matérielle a été isolée dans un adaptateur sans contaminer les autres couches.

RÉSULTAT OBSERVÉ

Après stabilisation des connexions et correction temporelle, les chaînes ``/scan0``, ``/scan1``, ``/scan_multi`` et ``/scan_fixed`` ont été observées autour de 7,14 Hz, sans trous ni erreurs d'extrapolation dans l'état propre documenté.

9.2 Ce que les essais ont changé dans la conception

Les essais n'ont pas seulement servi à corriger des anomalies. Ils ont confirmé plusieurs choix : garder les traitements critiques sur le robot, limiter les flux haute fréquence envoyés au web, pouvoir désactiver la caméra pendant la cartographie et conserver des paramètres ajustables sans reconstruire tout le système.

Sources projet : [R2] Architecture, sections de validation ; [R5] ROBOT.md ; [R6] Difficultés robot.

10. Contraintes, risques et limites

Risque	Effet possible	Réponse actuelle	Reste à faire
Charge du Jetson Nano	Retards, topics perdus, navigation instable.	Services limités, fréquences adaptées, caméra arrêtée selon le mode.	Mesures continues et budgets CPU par mode.
Réseau Wi-Fi	Robot déclaré à tort disponible ou commande retardée.	Last Will, reconnexion, état de présence.	Tests de coupure et expiration explicite des commandes.
Horloges et timestamps	Scans et TF incompatibles.	Restamping des LiDAR et synchronisation de l'hôte.	Surveillance automatique du décalage.
Saisie d'objet	Mission bloquée ou objet perdu.	Objet simple, couleur cible, reprise et retour d'erreur.	Calibration, détection de prise et essais répétés.
Sécurité physique	Mouvement inattendu ou arrêt insuffisant.	STOP logiciel, règles d'essai, contrôle humain.	Arrêt matériel, analyse de risques et conformité.
Dette documentaire	Écarts entre code, topics et UML.	Architecture canonique et contrat MQTT.	Génération ou vérification automatisée des schémas.

10.1 Limites du prototype

RobLaude démontre une architecture et plusieurs fonctions sur un robot réel ; il ne constitue pas encore un produit exploitable sans supervision dans un ERP. La robustesse de la saisie, la gestion des personnes et obstacles dynamiques, la disponibilité réseau et la sûreté de l'arrêt demandent davantage d'essais. La carte et les points doivent aussi être adaptés à chaque bâtiment.

Le projet n'a pas mesuré le nombre de missions quotidiennes, le coût de maintenance ou le temps réellement économisé par le personnel. Ces données sont indispensables avant une décision de déploiement. La prochaine étape produit doit donc être une expérimentation encadrée, pas une généralisation immédiate.

10.2 Conséquences des arbitrages

Le découplage améliore la maintenabilité mais multiplie les contrats à surveiller. MQTT absorbe des différences de rythme, mais ajoute un broker et des règles de QoS. ROS 2 accélère la construction de la navigation, mais demande une maîtrise des transformations et du temps. Docker rend l'environnement reproductible, au prix d'une consommation supplémentaire sur une machine limitée. Ces coûts sont acceptables pour le prototype parce qu'ils évitent une réécriture complète de briques complexes.

11. Axes d'amélioration et évolutivité

11.1 Priorités à court terme

Priorité	Action	Critère de validation
1	Formaliser un scénario de test de coupure réseau et d'expiration des commandes.	Aucune commande ancienne exécutée après une reconnexion prolongée.
2	Stabiliser la perception et la saisie sur un objet de référence.	Série d'essais répétables avec taux de réussite documenté.
3	Ajouter une surveillance des fréquences ROS, de l'horloge et de la charge CPU.	Alerte avant dégradation de la navigation.
4	Séparer clairement arrêt fonctionnel et arrêt de sécurité matériel.	Analyse de risques et dispositif physique testé.
5	Aligner automatiquement spécification MQTT, types backend et documentation.	Détection en CI d'une divergence de contrat.

11.2 Évolution multi-robot

L'arborescence MQTT contient déjà `robotId` et le modèle de données représente plusieurs robots. L'évolution ne consiste donc pas à dupliquer l'application, mais à ajouter une stratégie d'affectation : disponibilité, proximité, batterie et capacités. Le backend resterait l'autorité qui choisit un robot, tandis que chaque bridge ne recevrait que son sous-arbre de commandes.

11.3 Déploiement dans plusieurs bâtiments

Chaque bâtiment nécessite sa propre carte, ses points et une procédure de validation. Les entités de cartographie déjà présentes constituent une base, mais il faudrait associer clairement utilisateurs, robots, cartes et sites. Les contraintes d'accessibilité et de sécurité devraient être réévaluées pour chaque environnement.

11.4 Pérennité technique

La pérennité dépend moins du maintien éternel d'une version de React ou ROS 2 que de la stabilité des frontières. Le frontend peut évoluer si l'API reste contractuelle. Le broker peut être remplacé si le contrat de messages est conservé. Un nouveau robot peut être intégré derrière un bridge adapté. Cette indépendance limite le coût des migrations futures.

DÉCISION PROPOSÉE

Avant d'ajouter des fonctions visibles, investir dans la preuve : tests de reprise réseau, mesures de charge, validation répétée de la saisie et traçabilité automatique des contrats.

12. Traçabilité des besoins

Cette matrice relie le besoin initial à une réponse de conception, à une preuve disponible et à la limite connue. Elle évite de confondre intention, code et validation terrain.

Besoin	Réponse de conception	Preuve	Limite actuelle
Créer une mission simplement	PWA responsive et API métier.	Parcours et code web présents.	Validation utilisateur PMR à élargir.
Connaître l'avancement	MQTT vers backend puis WebSocket vers PWA.	Flux temps réel implémenté.	Dépendance au réseau local.
Naviguer dans le bâtiment	SLAM Toolbox, TF et Nav2.	Cartographie et navigation réelles documentées.	Carte à produire pour chaque site.
Éviter les obstacles	LiDAR et costmaps Nav2 sur `/scan_fixed`.	Chaîne LiDAR stable observée.	Tests avec public et obstacles dynamiques à renforcer.
Arrêter le robot	Commande dédiée et état de pause.	Flux logiciel présent.	Pas une chaîne d'arrêt certifiée.
Récupérer un objet	Caméra, détection, bras et pince.	Actionnement testé ; briques présentes.	Fiabilité de bout en bout non démontrée.
Administrer plusieurs robots	Topics par robot et modèle de données.	Architecture compatible.	Affectation multi-robot à développer.
Maintenir la solution	Composants séparés et contrats versionnés.	Packages, tests et documentation.	Certaines docs historiques ont divergé.

12.1 Lecture au regard du référentiel

Compétence	Éléments apportés
C1.1	Contexte ERP, acteurs, enjeux métier, objectifs hiérarchisés, retombées attendues et évolutions.
C1.2	Comparaison PWA/native, REST/WebSocket, MQTT/ROS direct, traitement embarqué/déporté ; conséquences exposées.
C1.3	Architecture en composants, contrats MQTT, données, quatre diagrammes, spécifications et évolutivité.

13. Conclusion

RobLaude répond à un besoin concret par une architecture volontairement découpée. Le projet ne réduit pas la robotique à une interface web, et il n'expose pas non plus le fonctionnement interne de ROS 2 au système métier. Chaque couche possède un rôle lisible : l'utilisateur formule une demande, le backend la valide et la conserve, MQTT assure le passage entre les environnements, puis le robot exécute et rend compte.

Les essais sur le robot réel ont validé la navigation, la cartographie et plusieurs flux essentiels. Ils ont aussi montré les limites du matériel et l'importance du temps dans une chaîne LiDAR/TF. La récupération autonome d'objet reste l'axe le plus incertain. Le reconnaître ne diminue pas la valeur du projet : cela permet de proposer une suite de travail réaliste et mesurable.

Le choix recommandé est donc de conserver ce socle, de renforcer les tests de sûreté et de reprise, puis de stabiliser la saisie avant toute extension fonctionnelle. L'architecture actuelle laisse ensuite une voie crédible vers plusieurs robots et plusieurs bâtiments.

BILAN

Une conception réussie n'est pas celle qui promet tout. C'est celle qui rend visibles ses choix, ses preuves, ses limites et la manière dont elle pourra évoluer.

Sources et références

Sources internes au projet

- [R1] RobLaude, Cahier des charges v1.2, docs/CDC_Roblaude_v1.2.md.
- [R2] RobLaude, Architecture et mémoire projet, docs/architecture.md.
- [R3] RobLaude, Roadmap v1.4, docs/ROADMAP.md.
- [R4] RobLaude, Spécification MQTT, docs/mqtt-spec.md.
- [R5] RobLaude, Faits robot sourcés, docs/ROBOT.md.
- [R6] RobLaude, Difficultés rencontrées sur le robot, docs/DIFFICULTES_ROBOT.md.
- [R7] Dépôt RobLaude, branche feat/demo-soutenance, code frontend, backend et packages ROS 2.

Références techniques externes

- [E1] MDN Web Docs, Making PWAs installable, https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable, consulté en août 2026.
- [E2] OASIS / MQTT.org, MQTT Specification, <https://mqtt.org/mqtt-specification/>, consulté en août 2026.
- [E3] Open Robotics, ROS 2 Humble - Interfaces: topics, services, actions, <https://docs.ros.org/en/humble/Concepts/Basic/Interfaces-Topics-Services-Actions.html>, consulté en août 2026.
- [E4] Nav2, documentation officielle, <https://docs.nav2.org/>, consultée en août 2026.

Note méthodologique

Les informations contradictoires entre les premiers UML et les sources récentes ont été tranchées à partir de l'architecture canonique, de la spécification MQTT et du code de la branche étudiée. Les anciennes mentions de caméra RealSense et de topics ``robot/...`` n'ont pas été conservées lorsque le projet documentait respectivement une Orbbec DaBai DCW2 et l'arborescence ``roblaude/{robotId}/...``.